

An In-Depth Comparison of Efficiency of Various Sorting Algorithms

Doc Holloway
Computer Science
Washington State
University Tri-
Cities
doc.holloway@wsu.edu

Jase Schab
Computer Science
Washington State
University Tri-
Cities
jase.schab@wsu.edu

Raymond Morrow
Cyber Security
Washington State
University Tri-
Cities
raymond.morrow@wsu.edu

Chloe Hinkley
Computer Science
Washington State
University Tri-
Cities
c.hinkley@wsu.edu

Computer Science 350: Algorithm Analysis
Dr. Asma Jodeiri Akbarfam

Abstract – In the world of programming and computer science, determining the most optimal algorithm for a particular problem is a foundational part of most tasks. Our team took an experimental approach of comparing the runtime and memory efficiency of various sorting algorithms to determine which has the most optimal performance. Using Valgrind's massif tool and the Chrono library in C++, we measured each algorithm's performance and were able to quantify the effective performance of Quick Sort, Merge Sort, Bogo Sort, and Bubble Sort to determine which algorithm had the best performance overall across various data types. The results of our experimental process concluded that Quick Sort was the most efficient algorithm overall for its optimal use of time and memory resources. In addition to Quick Sort, the other three algorithms were successfully ranked based on their memory and runtime performances to display a clear hierarchy of algorithm efficiency. Pulling from other related works and developing our own experimental process, we can confidently determine and discuss the ranking of these four algorithms.

I. INTRODUCTION

Finding the best solution to an existing problem is a key skill and procedure in the field of computer science. To facilitate the process of determining if a certain function will perform better than another to solve a programming problem, methods of algorithmic analysis are used. Two of the most critical points of analyzing an algorithm's performance for a certain problem are analyzing its runtime to complete a task and how much memory space it requires to operate. Finding an algorithm that most efficiently uses computer resources in an optimal time is critical for computer networks to ensure all the components of complex network systems perform at ideal times and in the correct order, without overloading its existing memory resources.

The experimental work of our team paid particular focus to the importance of time and space complexity for algorithms to determine which algorithm, out of a selected set, would perform most optimally. Selecting the algorithms Quick Sort,

Merge Sort, Bogo Sort, and Bubble Sort, we set out to determine each algorithm's runtime and memory usage, so that we could determine which algorithm would most optimally perform overall. As an additional measure of comparison, we set out to determine how the different algorithms would also perform based on the input data type provided to each algorithm, to determine if certain algorithms are better for certain data types. Our research and experiments aim to ultimately determine which algorithms perform the best overall, and how their performance varies across various data types.

II. BACKGROUND

The problem of sorting a list of items from least to greatest has several solutions depending on the items themselves, the constraints they can get around, and more. For this project, we aim to focus on 4 different sorting algorithms that have already been well-introduced: Bubble Sort, Merge Sort, Quick Sort, and Bogo Sort.

The premise of Bubble Sort is that values are repeatedly moved, one step at a time, to the correct position. This leaves the appearance of values "bubbling up" into their correct positions. Its worst-case runtime is $O(N^2)$, and because of this, it's recommended that data sizes using this sorting method generally remain under 1,000 [1].

On the other hand, Merge Sort is seen as considerably faster because it adopts a divide and conquer algorithm with fewer movements and swaps. It starts with a splitting phase, which separates the list into two based on a predetermined pivot, then this is repeated recursively and merged again [1].

Quick Sort acts in a very similar manner to Merge Sort, except it stays in the original array instead of creating two different subarrays. Quick Sort is better in memory use, but it

can still easily be improved to be even faster than Merge Sort [2].

Finally, the most curious and non-standard algorithm we are testing is Bogo Sort. Bogo Sort is probability-based, and simply repeatedly shuffled until the exact correct order is found. The average case runtime is $\Theta(N \cdot N!)$, which rapidly and easily increases with even small increases in data size. Because of this, Bogo Sort is frequently interpreted as an example of inefficient programming [3].

For this project, we aim to run tests using various data sizes to observe the versatility of each algorithm. We will be using two main metrics: runtime and memory usage. This way, efficiency can be visualized both in terms of how quickly the algorithm can be expected to run, and how much memory and power the algorithm can be expected to take. We expect our contribution to be more significant in terms of memory, since we found in our research that runtime is currently more well-studied for the given algorithms.

III. RELATED WORK

Sorting algorithm performance has been studied thoroughly, and a few recurring findings shaped what we chose to measure. Knuth's *The Art of Computer Programming* (1998) established the theoretical complexity baselines most people start from: Quick Sort at $\Theta(n \log n)$ average-case, Bubble Sort at $O(n^2)$ and we use those bounds to sanity-check our empirical results [4]. A 2015 study by Oyelami compared Bubble Sort, Insertion Sort, Selection Sort, and Merge Sort on datasets from 1,000 to 100,000 elements. Merge Sort pulled ahead at larger sizes while Bubble Sort degraded fast [5]. Our experiment adds Quick Sort and Bogo Sort, and crucially, we measure memory allocation alongside runtime rather than treating time as the only metric worth tracking.

Estivill-Castro and Wood (1992) looked at adaptive sorting behavior and found that Bubble Sort can approach $O(n)$ on nearly sorted data [6]. That motivated us to acquire good averages on our testing data to account for this possibility in randomly sorted data. More recent benchmarking work in ACM conference proceedings tends to show Quick Sort outperforming Merge Sort in practice, largely because of cache locality, even when both share the same asymptotic complexity [7]. That's a claim we tested directly in our experiment.

On the memory side, Sedgewick and Wayne's *Algorithms* (4th ed., 2011) note that Merge Sort's $O(n)$ auxiliary space requirement is a practical tradeoff, particularly at scale, even though it doesn't appear in the time-complexity analysis [8].

That framing informed our decision to track peak memory allocation as a first-class metric rather than an afterthought.

Bogo Sort shows up in academic literature mainly as a teaching example of non-deterministic algorithms. Black's *Dictionary of Algorithms and Data Structures* (NIST, 2004) document its expected runtime as $O((n+1)!)$ [8]. We included it on purpose, as it anchors the comparison and makes the cost of bad algorithmic choices concrete.

IV. SYSTEM DESIGN / METHODOLOGY

The experiment was straightforward in concept: run four sorting algorithms on the same datasets and measure how they behave. The four algorithms of Bubble Sort, Merge Sort, Quick Sort, and Bogo Sort were chosen to span a broad range of approaches, from brute-force randomization to divide-and-conquer, so the performance gaps would be meaningful.

Each algorithm ran against randomly generated sets of values for each of the four tested data types. These data types include integers, floats, characters, and strings. We tracked three metrics: execution time across multiple trials, peak memory allocation during each run, and spatial overhead, particularly for Merge Sort, which requires $O(n)$ auxiliary space that the other algorithms did not. Memory tracking matters here because time complexity alone does not tell the full story of algorithmic performance. Merge Sort and Quick Sort share the same asymptotic time bound, but their memory footprints differ in ways that matter in constrained environments. Bubble Sort and Bogo Sort are negligible in memory but show how badly time can scale when the algorithm is wrong for the job. Figure 1 below shows how each algorithm operates structurally.

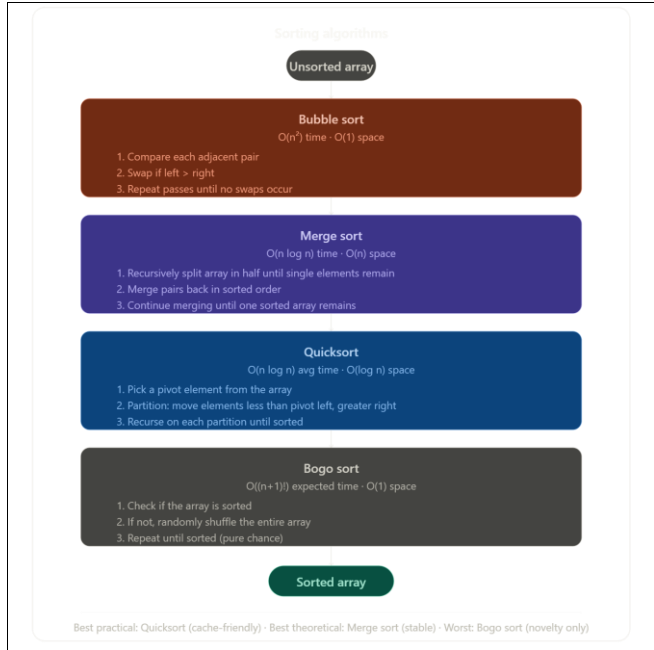


Fig. 1. System Design Figure

Bogo and Bubble Sort define the inefficient end of the spectrum. Merge Sort and Quick Sort show what divide-and-conquer buys you in practice. Together, they let us check whether asymptotic expectations hold on real hardware — and whether the memory costs that theory glosses over show up in measurements.

V. IMPLEMENTATION / EXPERIMENT SETUP

For the given project, several implementations and mechanisms were used. Specifically, four unique functions were written in C++ 20, and implemented in the Visual Studio IDE. Each one of these functions is dedicated to a different sorting algorithm tested and is formatted as a template function, so it can sort any primitive data type. As stated previously, we focused on integers, floats, strings, and characters. All of these can be found in Appendix A.

To test the algorithms, we created a separate test function in C++ 20 that could run all the created sorting algorithms. This test would create a vector of a predetermined amount of each type tested, populated with random entries of the given type. Then, the function repeatedly used the now() function in the Chrono library to note the current time at the start and end of running each sorting algorithm. This tracking process was repeated for data amounts from 100 to 1,000,000. Due to the immense average-case runtime of Bogo Sort, tests relating to Bogo Sort had data amounts reduced to a range of 1 to 12 for runtime and 3 to 5 for memory.

Similarly, Valgrind’s massif library was used to check and note memory usage across different data sizes and data types across all the algorithms. This testing was carried out directly in WSL command terminals, and we used looping commands to collect average memory usage across various test runs.

VI. RESULTS

Using our listed testing tools, we successfully ran ten cases of runtime and memory analysis for each algorithm with each of the four data types in the size range of 100 – 1 million, except for Bogo Sort whose testing required significantly smaller testing ranges of data size due to the algorithm’s poor performance. In figures 2-4 below, the time and memory results for each algorithm are displayed, specifically compared to each other based on their performance with different data types at the various data sizes.

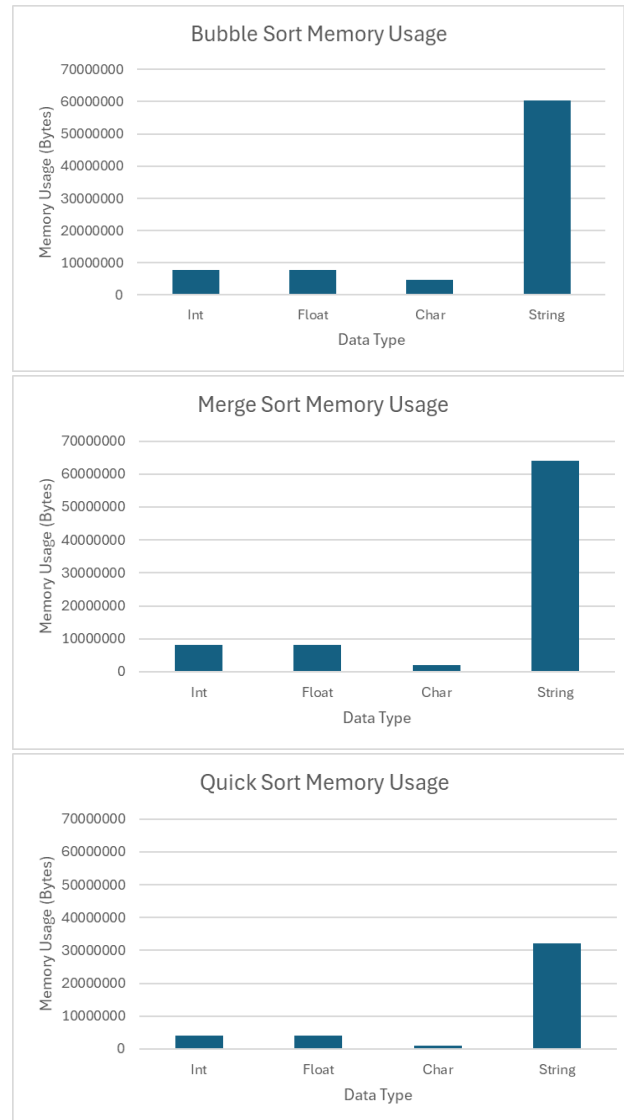


Fig. 2. Graphs of Average Merge, Quick, and Bubble Sort Memory Performance

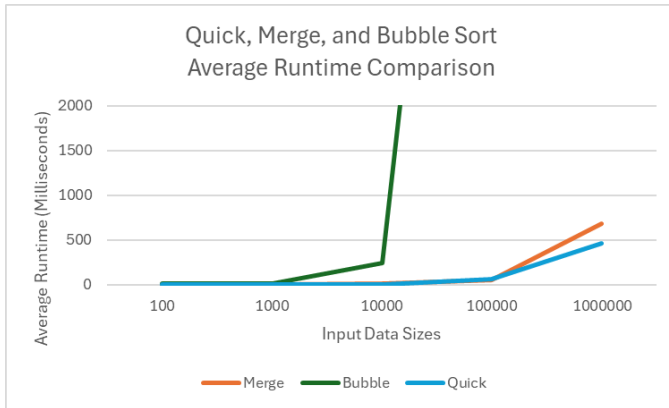


Fig. 3. Graph Comparison of Bubble, Merge, and Quick Sort Average Runtimes for Integer Data

Data size/Data Type	Int	Float	Char	String
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0
4	0	0	0	0
5	0	0	0	0
6	0	0	0	0
7	6	0	1	5
8	35	55	12	41
9	223	93	191	40
10	9700	2017	1307	3658
11	7382	27565	13286	101344
12	222419	1275431	7873	1524342

Data size/Data Type	Int	Float	Char	String
3	77860	77860	77833	78112
4	77872	77872	77836	78208
5	77884	77884	77839	78304

Fig. 4. Runtime and Memory Test Results for Bogo Sort, in Milliseconds and Bytes Respectively

Based on the testing results of the algorithms, Quick Sort had the lowest memory usage in all cases, and most often had the smallest runtimes. Quick sort used approximately 4 MB of memory and took 0.5 seconds to run on the average test case for 1 million integers. This stayed roughly consistent over most data types, with the exception of strings which took 32 MB of

memory to execute. Bubble Sort had the next best performance for memory at 8 MB of average memory, but Merge Sort had the next best runtime average overall with approximately 8000 seconds average runtime on integer data with 1 million variables. This is expected due to the lesser storage needed for Bubble Sort, and the smaller theoretical runtime of $O(n \cdot \log(n))$ for Merge Sort. On all metrics, Bogo Sort performed the worst out of all other algorithms due to scaling issues.

These results display a clear ranking of Quick Sort, then Bubble and Merge Sort, and finally Bogo sort for having the most optimal performance when compared to one another. This is reflected in the figures, and shows a clearly ranked relation between each algorithm's performance with various data sizes and types.

The most apparent flaw in these results comes from the inclusion of Bogo Sort. Its data serves as a useful baseline for a worst-case sorting algorithm that may never solve, but it is inefficient to the degree that its data cannot be seriously compared to the other algorithms. This creates issues when comparing the algorithm's performances on similar metrics to the other algorithms, and solidly always places Bogo Sort at the bottom tier of performance within the four test algorithms. Aside from the issues presented by Bogo Sort, no further adjustments or issues emerged in testing, based on our updated criteria for testing these four algorithms using memory and runtime data.

VII. DISCUSSION

The experiment results show clear performance differences between sorting algorithms because our team evaluated both their runtime and memory consumption. Quick Sort overall showed superior performance than Bubble, Merge, and Bogo Sort throughout all experiments, which tested different types of data and various input sizes. The results match expected theoretical complexity results, although they demonstrate important operational differences between the various algorithms.

Quick Sort achieved its best performance results because its average-case runtime performs well while its in-place execution method uses less memory. Merge Sort needed extra memory resources to create temporary arrays which it used during its merge operations, although it maintained good runtime performance. The Massif results showed that Merge Sort needed more memory space because it required additional space to split the array into repeatedly smaller subarrays.

Bubble Sort showed severe performance restrictions when faced with larger datasets. The algorithm uses little extra

memory because it works in-place, but its dual runtime operation made it unusable with vast datasets. Bubble Sort needs either extended processing time to finish work, or it needs time limits to stop testing. The algorithm exists as an unsuitable option for processing large data volumes.

Bogo Sort showed the most severe performance breakdown among all tested algorithms. The algorithm needed small input data to function because its runtime took factorial time to complete. The test results showed that Bogo Sort needed small input data to operate properly because all other input sizes led to extremely long runtime waits, which needed time limits to stop the program. These results showed constant memory usage because small input sizes made baseline program memory usage create the main test results.

VIII. CONCLUSION

In summary, our team was able to successfully implement the sorting functions and the test functions for both memory and runtime to find the most optimally performing algorithm to be Quick Sort. All completed testing and experimentation has led to this conclusion, with Merge Sort and Bubble Sort serving as acceptable seconds for runtime and memory usage respectively. Bogo sort, as expected, performed significantly worse than all other algorithms.

Generally speaking, our first thoughts were proven correct when we found that Merge Sort and Quick Sort were faster on all data types than Bubble Sort and Bogo Sort. We hypothesized this because Bubble Sort has an excessive amount of “navigation,” or moving access points in the given data vector, and Bogo Sort’s chance of shuffling into the correct order is extremely minimal. Secondly, we found that Bubble Sort could not efficiently sort a list of 1,000,000 strings, as the runtime took over several hours to complete. Continuing from this, we also found that with all the tested algorithms, strings took the longest to sort. We predicted this is due to the greater amount of data involved with a string, as opposed to a single character or float or integer.

In terms of memory, we found that Quick Sort was more notably the most efficient. This is followed by Bubble Sort, which is to be expected since it modifies itself without using a significant amount of extra data. Following that are Merge Sort and Bogo Sort, which took up the most memory. This is also to be expected, since Bogo Sort can potentially take a lot of computing power due to its need for numerous iterations.

In future work, we hope to discover new methods and metrics that can provide a further in-depth analysis of sorting algorithms. This way, we can paint a bigger picture on what each algorithm can and cannot do, and ultimately find a way to

solidify each algorithm’s best use cases in a more generalized sense.

REFERENCES

- [1] S. Nikpoor and I. Ahmad, 50 Algorithms Every Programmer Should Know: Tackle Computer Science Challenges with Classic to Modern Algorithms in Machine Learning, Software Design, Data Systems, and Cryptography, 2nd ed. Birmingham, U.K.: Packt Publishing, 2023.
- [2] T. Whitaker, “Interactive ebooks and course materials: A BOF for authors and instructors,” in *Proceedings of the 49th ACM Technical Symposium on Computer Science Education (SIGCSE '18)*, New York, NY, USA: ACM, 2018, pp. 1103–1103.
- [3] H. Gruber, M. Holzer, and O. Ruepp, “Sorting the slow way: An analysis of perversely awful randomized sorting algorithms,” in *Fun with Algorithms*, Lecture Notes in Computer Science, vol. 4475. Berlin, Heidelberg: Springer, 2007, pp. 183–197.
- [4] D. E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998.
- [5] M. O. Oyelami, “A comparative study of sorting algorithms,” *International Journal of Computer Science and Information Security*, vol. 13, no. 7, pp. 1-7, 2015
- [6] V. Estivill-Castro and D. Wood, “A survey of adaptive sorting algorithms,” *ACM Computing Surveys*, vol. 24, no. 4, pp. 441-476, Dec. 1992.
- [7] J. L. Bentley and M. D. McIlroy, “Engineering a sort function,” *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249-1265, Nov. 1993
- [8] R. Sedgewick and K. Wayne's *Algorithms*, 4th ed., 2011
- [9] P.E. Black, “BogoSort,” in *Dictionary of Algorithms and Data Structures*. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2004.

APPENDIX A APPENDIX (OPTIONAL)

Project Materials Github: [JaseSchab/Algorithm-Efficiency-Comparison-CS350](https://github.com/JaseSchab/Algorithm-Efficiency-Comparison-CS350)

Below in figures 5-8 are some sample snippets of code used to perform experimentation and testing with this project.

```
int main(int argc, char* argv[])
{
    //For loop for 3 different sizes
    for (int size = 100; size <= 100000; size *= 10)
    {
        //Creation of various testing type vectors
        vector<int> intVector = getInts(size, 1);
        vector<float> floatVector = getFloats(size, 1);
        vector<char> charVector = getChars(size, 1);
        vector<string> stringVector = getStringss(size, 1);

        cout << "Testing Data Size: " << size << endl;

        cout << "Integer time" << endl;
        //Either change this function name, or change the name of your sorting function
        auto start = steady_clock::now();
        quickSort(intVector, 0, size-1);
        auto end = steady_clock::now();
        auto duration = duration_cast<milliseconds>(end - start);
        cout << "Time: " << duration.count() << " ms." << endl;
    }
}
```

Fig. 5. Test File Coding Snippet

```

// greater than pivot, so recurse on and leaving values equal to pivot in middle.
template<typename type>
pair<int, int> threeWayPartition(vector<type>& array, int start, int end)
{
    int leftPart = start;
    int index = start;
    int rightPart = end;

    int mid = start + (end - start) / 2;

    type pivot = array[mid];

    //For all unvisited values
    for (index; index <= rightPart; index++)
    {
        //If less than pivot, swap to beginning of equals section
        if (array[index] < pivot)
        {
            swap(array[index], array[leftPart]);
            leftPart++;
        }
        //If greater than, swap to end of equals section
        else if (array[index] > pivot)
        {
            swap(array[index], array[rightPart]);
            rightPart--;
            index--;
        }
    }

    return { leftPart, rightPart };
}

```

Fig. 6. Quick Sort Function Snippet

```

template <typename T>
void bubbleSort(vector<T>& arr)
{
    int n = static_cast<int>(arr.size());
    for (int i = 0; i < n - 1; i++)
    {
        bool swapped = false;
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break; // early exit if already sorted
    }
}

```

Fig. 7. Bubble Sort Function Snippet

```

template <typename T>
void mergeSort(vector<T>& arr, int left, int right)
{
    if (left >= right)
    {
        return;
    }

    int mid = left + (right - left) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

```

Fig. 8. Merge Sort Function Snippet

```

template<class T>
std::vector<T> bogoSort(std::vector<T> array)
{
    //newarray stores the randomized array
    std::vector<T> newarray;
    newarray.resize(array.size());

    //run until sorted
    while(true)
    {
        //repopulate array with randomized values from original array

        //temporary value to use for loop
        int temp = array.size();

        //for all values in original array, pick one at random, erase it from array, then push it onto newarray
        for (int i = 0; i < temp; i++)
        {
            int randint = rand() % array.size();
            newarray[i] = array[randint];
            array.erase(array.begin() + randint);
        }

        //now need to check if its sorted

        //default to true
        bool sorted = true;

        //if a decrease is detected, array is not sorted and must loop
        for (int i = 0; i < (newarray.size() - 1); i++)
        {
            if (newarray[i] > newarray[i + 1])
            {
                sorted = false;
            }
        }

        //if newarray is sorted, break out and return
        if (sorted == true)
        {
            break;
        }

        //at this point, array is confirmed unsorted and we must reset things

        //repopulate array using preserve
        for (int i = 0; i < newarray.size(); i++)
        {
            array.push_back(newarray[i]);
        }
    }

    //return sorted array
    return newarray;
}

```

Fig. 9. Bogo Sort Function Snippet